

Instituto Tecnológico de Costa Rica

Lenguajes de Programación

Tarea Corta 2

Estudiante: Luis Carlos Trejos Rivera

Carné: 202243786

Profesor: Allan Rodríguez Davila

Semestre I, 2026

15 de abril de 2026

Manual de Usuario

Requisitos

- SML of New Jersey v110.99.x instalado
- Sistema operativo: Windows, Linux o macOS
- Archivo matricula.csv en la misma carpeta que los programas

Compilación y Ejecución del Creador

1. Abrir terminal en la carpeta del proyecto
2. Ejecutar:

```
sml
```

3. Dentro del REPL escribir:

```
use "creador.sml";  
main();
```

El sistema mostrara el menú de administración con las siguientes opciones:

1. **Agregar nueva matricula:** solicita carné, nombre, código de curso, créditos y costo por crédito.
2. **Limpiar todo el catálogo:** elimina todos los registros y conserva solo el encabezado del CSV.
3. **Salir:** cierra el programa.

Compilación y Ejecución del Analizador

1. Abrir terminal en la carpeta del proyecto
2. Ejecutar:

```
sml
```

3. Dentro del REPL escribir:

```
use "analizador.sml";  
main();
```

4. Ingresar la ruta del archivo CSV cuando se solicite (ej: `matricula.csv`)

El sistema mostrara el menú de análisis con las opciones a, b, c, d, e y s (salir).

Formato del archivo CSV

El archivo debe tener el siguiente encabezado y estructura:

```
carnet_estudiante,nombre,curso,creditos,costo_credito  
200440117, Juan Perez,IC4301,4,15000.0
```

Cada línea representa una matrícula. Un estudiante puede tener múltiples líneas.

Pruebas de Funcionalidad

Creador:

Inicio de la aplicación:

```
PS C:\Users\luist\OneDrive\Escritorio\Tarea02> sml  
  
C:\Users\luist\OneDrive\Escritorio\Tarea02>REM sml.bat  
  
C:\Users\luist\OneDrive\Escritorio\Tarea02>REM  
  
C:\Users\luist\OneDrive\Escritorio\Tarea02>REM Copyright 2020 The Fellowship of SML/NJ (http://www.smlnj.org)  
  
C:\Users\luist\OneDrive\Escritorio\Tarea02>REM All rights reserved.  
  
C:\Users\luist\OneDrive\Escritorio\Tarea02>REM  
  
C:\Users\luist\OneDrive\Escritorio\Tarea02>REM The standard driver for SML/NJ under the new runtime system  
  
C:\Users\luist\OneDrive\Escritorio\Tarea02>REM  
Standard ML of New Jersey (32-bit) v110.99.4 [built: Tue Aug 01 11:26:46 2023]  
- use "creador.sml";  
[opening creador.sml]  
[autoloading]  
[library $SMLNJ-BASIS/basis.cm is stable]  
[library $SMLNJ-BASIS/(basis.cm):basis-common.cm is stable]  
[autoloading done]  
type matricula = string * string * string * int * real  
val archivoPorDefecto = "matricula.csv" : string  
val formatoCSV = fn : string * string * string * int * real -> string  
val archivoTerminaEnNewline = fn : string -> bool  
val agregarRegistro = fn  
  : string -> string * string * string * int * real -> unit  
val limpiarCatalogo = fn : string -> unit  
val leerEntero = fn : string -> int  
val leerReal = fn : string -> real  
val leerString = fn : string -> string  
val menuCreador = fn : string -> unit  
val main = fn : unit -> unit  
val it = () : unit  
- main();  
=== PROGRAMA CREADOR DE MATRICULAS ===  
Usando archivo por defecto: matricula.csv  
  
=== ADMINISTRACION DE MATRICULAS ===  
1. Agregar nueva matricula  
2. Limpiar todo el catalogo  
3. Salir  
Opcion: █
```

Registro de nueva matricula:

```
=== PROGRAMA CREADOR DE MATRICULAS ===  
Usando archivo por defecto: matricula.csv
```

```
=== ADMINISTRACION DE MATRICULAS ===
```

1. Agregar nueva matricula
2. Limpiar todo el catalogo
3. Salir

Opcion: 1

Carnet: 2021784619

Nombre: Prueba

Codigo del curso: IP1111

Creditos: 7

Costo por credito: 12000

Matricula agregada correctamente.

```
=== ADMINISTRACION DE MATRICULAS ===
```

1. Agregar nueva matricula
2. Limpiar todo el catalogo
3. Salir

Opcion: █

matricula.csv > data

	carnet_estudiante	nombre	curso	creditos	costo_credito
12	2023123456	Laura Mora	IC2301	3	10000.0
13	2023123459	Laura Mora	IC3301	3	17000.0
14	2020123460	Pedro Vargas	IC2201	4	15500.0
15	2020123460	Pedro Vargas	IC3002	4	18000.0
16	2020123460	Pedro Vargas	IC3401	4	18000.0
17	2022437816	Luis Trejos	IC4301	8	80000.0
18	2021784619	Prueba	IP1111	7	12000.0
19					

Prueba de limpiar todo:

```
matricula.csv > data
1 carnet_estudiante,nombre,curso,creditos, costo_credito
2
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS QUERY RESULTS

```
PS C:\Users\luist\OneDrive\Escritorio\Tarea02> sml
```

```
val limpiarCatalogo = fn : string -> unit
val leerEntero = fn : string -> int
val leerReal = fn : string -> real
val leerString = fn : string -> string
val menuCreador = fn : string -> unit
val main = fn : unit -> unit
val it = () : unit
- main();
=== PROGRAMA CREADOR DE MATRICULAS ===
Usando archivo por defecto: matricula.csv
```

```
=== ADMINISTRACION DE MATRICULAS ===
```

1. Agregar nueva matricula
2. Limpiar todo el catalogo
3. Salir

Opcion: 1

Carnet: 2021784619

Nombre: Prueba

Codigo del curso: IP1111

Creditos: 7

Costo por credito: 12000

Matricula agregada correctamente.

```
=== ADMINISTRACION DE MATRICULAS ===
```

1. Agregar nueva matricula
2. Limpiar todo el catalogo
3. Salir

Opcion: 2

Catalogo limpiado (solo queda la cabecera).

```
=== ADMINISTRACION DE MATRICULAS ===
```

1. Agregar nueva matricula
2. Limpiar todo el catalogo
3. Salir

Opcion: █

Analizador:

Inicio de Analizador:

```
- use "analizador.sml";
[opening analizador.sml]
[autoloading]
[library $SMLNJ-LIB/Util/smlnj-lib.cm is stable]
[autoloading done]
analizador.sml:140.18 Warning: calling polyEqual
analizador.sml:133.18 Warning: calling polyEqual
analizador.sml:124.18 Warning: calling polyEqual
analizador.sml:116.18 Warning: calling polyEqual
analizador.sml:117.43 Warning: calling polyEqual
analizador.sml:105.56 Warning: calling polyEqual
analizador.sml:74.18 Warning: calling polyEqual
analizador.sml:75.43 Warning: calling polyEqual
analizador.sml:54.18 Warning: calling polyEqual
type matricula = string * string * string * int * real
val leerCSV = fn : string -> (string * string * string * int * real) list
val leerEntero = fn : string -> int
val leerReal = fn : string -> real
val leerString = fn : string -> string
val rankingCursos = fn : ('a * 'b * string * int * real) list -> unit
val cursosMas5Estudiantes = fn : ('a * 'b * string * 'c * 'd) list -> unit
val buscarPorEstudiante = fn
  : (string * string * string * int * real) list -> unit
val cursosPorCreditos = fn : ('a * 'b * string * int * 'c) list -> unit
val resumenGeneral = fn : (string * string * string * int * real) list -> unit
val menuAnalisis = fn : (string * string * string * int * real) list -> unit
val main = fn : unit -> unit
val it = () : unit
- main();
=== ANALIZADOR DE MATRICULAS ===
Ruta del archivo CSV (ej. matricula.csv): matricula.csv
Se cargaron 17 matriculas.
```

Prueba A:

```
=== MENU DE ANALISIS ===
a) Ranking de cursos por ingreso (con filtro)
b) Cursos con mas de 5 estudiantes
c) Buscar matriculas por estudiante
d) Listar cursos por numero de creditos
e) Resumen general
s) Salir
Opcion: a

--- Ranking de cursos por ingreso total ---
Ingrese monto minimo: 1000
Ingrese monto maximo: 100000
IC2101 - 96000.0
IP1111 - 84000.0
IC3401 - 72000.0
IC3202 - 64000.0
IC2201 - 62000.0
IC1400 - 60000.0
IC2401 - 60000.0
IC1802 - 54000.0
IC3101 - 52500.0
IC3301 - 51000.0
IC2501 - 49500.0
```

Prueba B:

```
=== MENU DE ANALISIS ===
a) Ranking de cursos por ingreso (con filtro)
b) Cursos con mas de 5 estudiantes
c) Buscar matriculas por estudiante
d) Listar cursos por numero de creditos
e) Resumen general
s) Salir
Opcion: b

--- Cursos con mas de 5 estudiantes ---
Ningun curso supera los 5 estudiantes.
```

Prueba C:

```
=== MENU DE ANALISIS ===
```

- a) Ranking de cursos por ingreso (con filtro)
 - b) Cursos con mas de 5 estudiantes
 - c) Buscar matriculas por estudiante
 - d) Listar cursos por numero de creditos
 - e) Resumen general
 - s) Salir
- Opcion: c

Ingrese carnet exacto o parte del nombre: Luis Trejos

Resultados:

2022437816 | Luis Trejos | IC4301 | 8 creditos | 80000.0/credito

Prueba D:

```
=== MENU DE ANALISIS ===
```

- a) Ranking de cursos por ingreso (con filtro)
 - b) Cursos con mas de 5 estudiantes
 - c) Buscar matriculas por estudiante
 - d) Listar cursos por numero de creditos
 - e) Resumen general
 - s) Salir
- Opcion: d

Ingrese numero de creditos: 4

--- Cursos con 4 creditos ---

IC1400
IC2001
IC2401
IC3202
IC2201
IC3002
IC3401

Prueba E:

```

=== MENU DE ANALISIS ===
a) Ranking de cursos por ingreso (con filtro)
b) Cursos con mas de 5 estudiantes
c) Buscar matriculas por estudiante
d) Listar cursos por numero de creditos
e) Resumen general
s) Salir
Opcion: e

=== RESUMEN GENERAL ===
1. Cantidad de estudiantes por curso:
   IC1400: 1 estudiantes
   IC2001: 2 estudiantes
   IC3002: 2 estudiantes
   IC1802: 1 estudiantes
   IC2101: 2 estudiantes
   IC3101: 1 estudiantes
   IC2401: 1 estudiantes
   IC3202: 1 estudiantes
   IC2501: 1 estudiantes
   IC3301: 1 estudiantes
   IC2201: 1 estudiantes
   IC3401: 1 estudiantes
   IC4301: 1 estudiantes
   IP1111: 1 estudiantes
2. Estudiante con mayor creditos: Juan Perez (2020123456) (12 creditos)
3. Estudiante con menor creditos: Prueba (2021784619) (7 creditos)
4. Curso con mayor ingreso total: IC4301 (640000.0)
5. Estudiante que genera mayor ingreso: Luis Trejos (2022437816) con 640000.0

```

Descripción del Problema

El sistema desarrollado en esta tarea busca gestionar y analizar un registro de matrículas universitarias utilizando el paradigma funcional en Standard ML (SML). El problema consiste en procesar datos almacenados en un archivo CSV, representarlos en memoria como listas de tuplas, y permitir al usuario realizar operaciones de consulta y administración sobre esos datos.

Se implementaron dos programas independientes: el Creador, que permite agregar nuevos registros y limpiar el catálogo, y el Analizador, que carga el archivo y ofrece un menú de opciones para analizar la información. Ambos hacen uso de los principios del paradigma funcional, incluyendo funciones de orden superior, listas y tuplas como estructuras de datos primitivas, y la ausencia de estado mutable.

Diseño del Programa

Estructura de datos

Cada matricula se representa como una tupla del tipo:

```
type matricula = string * string * string * int * real
```

Los campos corresponden a: carnet del estudiante, nombre, código del curso, cantidad de créditos y costo por crédito. El archivo CSV se carga en memoria como una lista de este tipo.

Algoritmos utilizados

- **Lectura del CSV:** se usa `TextIO.openIn` y `TextIO.inputLine` para leer el archivo línea por línea. Cada línea se tokeniza por comas con `String.tokens` y se convierte al tipo `matricula` mediante `pattern matching`.
- **Ranking de cursos (opción a):** se acumula el monto total por curso usando `List.foldl`, luego se ordena con `ListMergeSort.sort` y se filtra el rango con `List.filter`.
- **Cursos con más de 5 estudiantes (opción b):** se construye un mapa `curso->lista de carnets` con `List.foldl`, evitando duplicados con `List.exists`, y luego se filtra con `List.filter`.
- **Búsqueda por estudiante (opción c):** se aplica `List.filter` con una función que verifica carnet exacto con `(=)` o subcadena del nombre con `String.isSubstring`.
- **Cursos por créditos (opción d):** se aplica `List.filter` seguido de `List.map` para extraer solo el código del curso, y se eliminan duplicados con una función auxiliar recursiva.
- **Resumen general (opción e):** combina `List.foldl` para acumular créditos y montos por estudiante y por curso, `List.map` para extraer valores y calcular máximos y mínimos.

Librerías Usadas

Librería	Uso
TextIO	Lectura y escritura de archivos CSV y entrada del usuario desde consola.
String	Tokenización de líneas (<code>String.tokens</code>), búsqueda de subcadenas (<code>String.isSubstring</code>), prefijos (<code>String.isPrefix</code>).
List	Funciones de orden superior: <code>List.foldl</code> , <code>List.filter</code> , <code>List.map</code> , <code>List.app</code> , <code>List.find</code> , <code>List.exists</code> .
ListMergeSort	Ordenamiento del ranking de cursos por monto total de forma descendente.
Int / Real	Conversion entre tipos (<code>Int.fromString</code> , <code>Real.fromString</code> , <code>Int.toString</code> , <code>Real.toString</code>).

Análisis de Resultados

Objetivos alcanzados

- Agregar nuevos registros de matrícula al archivo CSV correctamente.
- Limpiar el catálogo conservando el encabezado del archivo.
- Ranking de cursos por ingreso total con filtro de rango.

- Identificación de cursos con más de 5 estudiantes distintos.
- Búsqueda de matrículas por carnet exacto o nombre parcial.
- Listado de cursos por cantidad de créditos.
- Resumen general con estadísticas de créditos e ingresos.
- Uso correcto de List.map, List.filter y List.foldl en ambos programas.

Objetivos no alcanzados

Todos los requerimientos funcionales indicados en las instrucciones fueron implementados y probados correctamente.

Observaciones

Durante el desarrollo se identificó que TextIO.appendOut no existe en SML/NJ; la función correcta es TextIO.openAppend. También se corrigió un error en el comparador de ListMergeSort.sort, que requiere una función que retorne bool y no order. Además, se agregó validación para que el archivo no genere registros pegados al final de la última línea al agregar una nueva matrícula.

Justificación de Funciones de Orden Superior

List.foldl

Se utiliza en el Analizador para acumular información a lo largo de la lista de matrículas. Por ejemplo, en rankingCursos se usa para construir una lista de (curso, monto Total) sumando el ingreso de cada matrícula. En resumen, General se usa tres veces: para contar estudiantes por curso, para sumar créditos por estudiante y para sumar ingresos por estudiante y por curso. Se eligió foldl porque permite recorrer la lista una única vez y acumular el resultado en un valor de salida, lo que es idiomático en programación funcional y evita estructuras mutables.

List.filter

Se usa para seleccionar subconjuntos de datos según criterios específicos. En rankingCursos filtra los cursos cuyo monto total cae dentro del rango ingresado por el usuario. En cursosMas5Estudiantes selecciona solo los cursos con más de 5 estudiantes distintos. En buscarPorEstudiante filtra las matrículas que coinciden con el carné o nombre buscado. En cursosPorCreditos filtra las matrículas con la cantidad de créditos indicada. Se eligió filter porque permite expresar el criterio de selección como una función pura sin necesidad de un ciclo imperativo.

List.map

Se utiliza para transformar cada elemento de una lista en otro valor. En cursosPorCreditos se usa para extraer únicamente el código del curso de cada tupla de matrícula. En resumenGeneral se usa para extraer los valores de créditos de la lista de (carné, créditos) y luego calcular el máximo y mínimo. Se eligió map porque permite aplicar una transformación uniforme a todos los elementos de una lista de forma declarativa, separando la lógica de transformación de la lógica de iteración.